

Multi-Source Candidate Data Transformer — Technical Design

Shrijan · shrijan327@gmail.com · Eightfold Engineering Intern (Jul–Dec 2026)

1. Pipeline

detect → **extract** → **normalize** → **merge (+confidence)** → **project-to-output (config)** → **validate**. Detect identifies source type from input shape (CSV columns, JSON keys, file extension). Extract is a per-source parser producing a flat list of raw facts: {candidate_key, field, value, source, method} — never the canonical object directly. Normalize converts raw values into fixed formats. Merge groups facts by candidate and resolves conflicts into one canonical record with provenance/confidence. Project reshapes the canonical record per a runtime config. Validate checks the projected output against the requested field types/required-ness before it leaves the system.

2. Canonical Schema & Normalized Formats

Schema matches the spec given: candidate_id, full_name, emails[], phones[], location{city,region,country}, links{linkedin,github,portfolio,other[]}, headline, years_experience, skills[{name,confidence,sources[]}], experience[{company,title,start,end,summary}], education[], provenance[{field,source,method}], overall_confidence. Formats: phones → E.164 (best-effort; assumes a default country dial code only when no other context exists); dates → YYYY-MM (day truncated; unparseable strings become null, never guessed); country → ISO-3166 alpha-2 via a lookup table; skills → canonical names via a synonym map (e.g. "js"/"node" → "JavaScript"/"Node.js"), falling back to title-cased raw value for unmapped terms.

3. Merge / Conflict-Resolution & Confidence

Match key: lowercase email when present, else lowercase full name (documented limitation — no fuzzy/phonetic matching). **Scalar fields** (name, headline, years, location): the highest-priority source wins, where priority is recruiter_csv > ats_json > recruiter_notes/resume/github < linkedin (structured/recruiter-verified data outranks free-text-derived data). **List fields** (emails, phones, skills): union across sources after normalization, deduplicated. **Confidence** per field = (priority of winning source / max priority), reduced 15% if sources actively disagreed on the value; overall_confidence = mean of populated per-field confidences. This keeps confidence interpretable and traceable back to provenance rather than an opaque learned score.

4. Runtime Config (Projection + Validation)

The canonical record is never mutated for output. A separate projection layer reads a config of {fields:[{path, from, type, required}], include_confidence, include_provenance, on_missing}. Each output field is resolved from a canonical path (supports dotted paths and a "list[].subfield" extraction syntax for re-shaping arrays). on_missing controls behavior per the spec: null keeps the key with a null value, omit drops the key, error raises and the candidate record is excluded from output (logged, not crashed). After projection, each value is type-checked against the declared type before being returned, so a malformed config or upstream bug surfaces immediately rather than silently shipping bad data.

5. Edge Cases

- Missing/garbage source file or row (e.g. malformed CSV row, unreadable notes file): extractor logs a warning and skips that row/file; the run continues — never crashes.
- Same person, conflicting values across sources (e.g. two different titles): highest-priority source wins for scalars, confidence is reduced to reflect disagreement, and provenance records which source won — nothing is silently merged into one "average" value.
- Same phone number formatted differently across sources (e.g. "9876543210" vs "+91 98765 43210"): normalized independently to E.164; if they don't collapse to an identical string post-normalization (e.g. ambiguous local vs. country-coded numbers) both are kept as separate list entries rather than guessed-merged — a known limitation called out in the README.
- Unparseable date/skill/value: normalizer returns null rather than inventing a value, per the explainability constraint ("unknown values become null, never invented").
- No source provides a particular candidate field at all (e.g. no education anywhere): field stays null/empty and is simply not present in provenance — overall_confidence is computed only over fields that were actually populated, so absent data doesn't silently drag the score down or up.

What I'd deliberately leave out under time pressure: fuzzy name-matching/entity resolution beyond exact email/name keys, an NLP-based resume parser (regex/keyword heuristics only for notes), live GitHub/LinkedIn API calls (designed for, not wired up, to keep the sample run deterministic and offline), and empirical scale testing beyond asserting the merge/normalize functions are pure and O(n).